BrewFlow Final Project

Teil 2 OLAP Star Schema

Projektdokumentation zum analytischen Datenmodell und zur Validierung des Warehouse-DDL

Ziel

Teil 2 überführt das operative BrewFlow-Datenmodell in ein analytisches Star-Schema. Dazu werden die Granularität der Faktentabelle, die Kennzahlen, fünf Dimensionen und die Strategie zur Behandlung von Änderungen festgelegt. Anschließend wird die Tabellenstruktur im Schema brewflow_warehouse angelegt und überprüft.

Die Struktur ist erfolgreich eingerichtet: Sechs Warehouse-Tabellen sind vorhanden, die Kundendimension besitzt die vorgesehenen SCD-Type-2-Spalten, die Faktentabelle verweist auf alle fünf Dimensionen und revenue wird automatisch berechnet. Die beiden Verkaufskanäle sind bereits befüllt.

Teil A Fakten und Dimensionen bestimmen

Aufgabenstellung Das Geschäftsereignis auf der niedrigsten Analyseebene sowie Kennzahlen und beschreibende Dimensionen festlegen.

Schritt Die OLTP-Bestellpositionen als Ausgangspunkt des analytischen Modells betrachten.

Aktion Die Granularität von fact_order_items ist eine Zeile pro Bestellposition aus order_items. Eine Position kann mehrere Stück eines Produkts enthalten. quantity misst die Menge; revenue misst den Positionsumsatz als quantity × unit_price. Der Einzelpreis wird aus der Bestellposition übernommen und bildet damit den Verkaufspreis der Transaktion ab.

OLTP-Quelle	Dimension	Beschreibung
customers	dim_customer	Kunde und Loyalty-Level
products	dim_product	Produkt und Kategorie
stores	dim_store	Filiale und Standort
orders.channel	dim_channel	Verkaufskanal
orders.order_date	dim_date	Kalenderdatum

Kontrolle Jede Position lässt sich einem Kunden, Produkt, Standort, Kanal und Datum zuordnen. order_id bleibt als Bestellreferenz in der Faktentabelle; order_item_id hält die Zuordnung zur Quellposition fest.

Ergebnis Eine Faktentabelle mit fünf Dimensionen bildet das Star-Schema. Die additiven Kennzahlen sind Menge und Umsatz; Einzelpreise werden nicht über Positionen summiert.

Teil B SCD Entscheidungen treffen

Aufgabenstellung Historische Loyalty-Level von Kunden erhalten und die Änderungsstrategie der übrigen Dimensionen festlegen.

Schritt Zwischen historisierten Kundenattributen und überschreibbaren Beschreibungsattributen unterscheiden.

Aktion dim_customer verwendet SCD Type 2 für loyalty_tier. Bei einem Levelwechsel wird die bisherige Version beendet und eine neue Zeile mit einem neuen customer_sk angelegt. customer_id verbindet die Versionen mit demselben OLTP-Kunden. Fakten verweisen auf die zum Bestelldatum gültige Version.

Beispiel: Ein späteres Upgrade von Sofia darf die Auswertung ihrer Bestellung 106 nicht rückwirkend einem neuen Loyalty-Level zuordnen. Dafür muss die Faktzeile den Schlüssel der damals gültigen Kundenversion behalten.

Spalte	Aufgabe
customer_sk	Surrogatschlüssel der Kundenversion
effective_from	Beginn der Gültigkeit
effective_to	Ende der Gültigkeit; bei offener Version NULL
is_current	Kennzeichen der aktuellen Version

Die drei Gültigkeitsfelder bilden zusammen mit customer_sk die vier vorgesehenen Strukturmerkmale. customer_sk ist zugleich der Primärschlüssel der Dimension.

dim_product und dim_store werden nach SCD Type 1 behandelt: Attribute werden aktualisiert, ohne eine historische Version anzulegen. Vorhandene Surrogatschlüssel bleiben dabei erhalten. dim_channel enthält feste Referenzwerte; dim_date wird als Kalendertabelle vorab erzeugt.

Kontrolle Die Tabellenstruktur erlaubt mehrere Versionen je customer_id und die Verknüpfung von Fakten mit einzelnen Versionen. Prüfung 3 bestätigt die vorgesehenen Spalten.

Ergebnis Die SCD-Strategie ist festgelegt. Das DDL stellt die Struktur bereit; das Beenden und Anlegen von Versionen sowie die zeitliche Zuordnung der Fakten müssen im Ladeprozess umgesetzt werden.

Teil C Warehouse DDL schreiben

Aufgabenstellung Das Warehouse-Schema und seine sechs Tabellen mit Schlüsseln, Beziehungen und automatischer Umsatzberechnung anlegen.

Schritt Das Schema aktivieren, zuerst die Dimensionen und danach die Faktentabelle erstellen. Die Verkaufskanäle mit den beiden Referenzwerten befüllen.

Aktion und SQL Die folgenden Anweisungen dokumentieren die ausgeführte Einrichtung. CREATE TABLE und die Kanal-INSERTs sind für die Erstinstallation vorgesehen.

```
-- Teil C Schritt 1 Warehouse Schema erstellen
CREATE SCHEMA IF NOT EXISTS brewflow_warehouse;
SET search_path TO brewflow_warehouse;

-- Kontrolle Aktives Schema
SELECT current_schema() AS active_schema;


-- Teil C Kundendimension mit SCD Type 2
CREATE TABLE dim_customer (
    customer_sk INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id INT NOT NULL,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    city VARCHAR(50),
    loyalty_tier VARCHAR(20),
    effective_from DATE NOT NULL,
    effective_to DATE,
    is_current BOOLEAN DEFAULT TRUE
);

-- Teil C Produktdimension
CREATE TABLE dim_product (
    product_sk INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    product_id INT NOT NULL,
    name VARCHAR(100),
    category VARCHAR(50),
    unit_price DECIMAL(6,2)
);

-- Teil C Filialdimension
CREATE TABLE dim_store (
    store_sk INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    store_id INT NOT NULL,
    name VARCHAR(100),
    city VARCHAR(50),
    country VARCHAR(50)
);
```

Teil C Weitere Dimensionen und Fakten

```
-- Teil C Kanaldimension
CREATE TABLE dim_channel (
    channel_sk INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    channel VARCHAR(20) NOT NULL,
    channel_group VARCHAR(50)
);

-- Teil C Verkaufskanäle befüllen
INSERT INTO dim_channel (channel, channel_group) VALUES
('in-store', 'Physical'),
('mobile-app', 'Digital');

-- Teil C Kalenderdimension
CREATE TABLE dim_date (
    date_sk INT PRIMARY KEY, -- Format YYYYMMDD
    calendar_date DATE NOT NULL,
    year INT,
    quarter INT,
    month INT,
    month_name VARCHAR(10),
    day_of_week VARCHAR(10),
    is_weekend BOOLEAN
);

-- Teil C Faktentabelle
-- Granularität Eine Zeile pro Bestellposition
CREATE TABLE fact_order_items (
    order_item_sk INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_item_id INT NOT NULL,
    order_id INT NOT NULL,
    customer_sk INT REFERENCES dim_customer(customer_sk),
    product_sk INT REFERENCES dim_product(product_sk),
    store_sk INT REFERENCES dim_store(store_sk),
    channel_sk INT REFERENCES dim_channel(channel_sk),
    date_sk INT REFERENCES dim_date(date_sk),
    quantity INT,
    unit_price DECIMAL(6,2),
    revenue DECIMAL(8,2)
        GENERATED ALWAYS AS (quantity * unit_price) STORED
);
```

Kontrolle Die Schemaabfrage lieferte active_schema = brewflow_warehouse. Die fünf Prüfungen in Teil D bestätigen Tabellen, Kanaldaten, SCD-Spalten, Fremdschlüssel und die generierte Umsatzspalte.

Ergebnis Das Warehouse-DDL ist umgesetzt. Die Strukturprüfungen sind bestanden; die Kalendergenerierung und die Befüllung der übrigen Dimensionen und Fakten gehören zum anschließenden Ladeprozess.

Teil D Prüfung 1 Warehouse Tabellen

Aufgabenstellung Die Existenz aller sechs Warehouse-Tabellen bestätigen.

Schritt Tabellennamen aus information_schema.tables für das Warehouse-Schema abfragen.

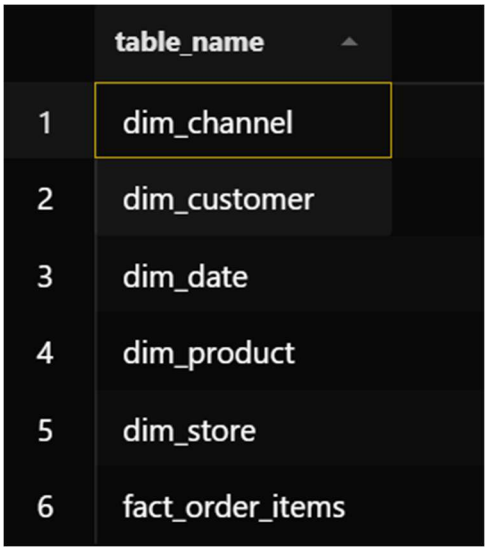
Aktion und SQL Die folgende Kontrollabfrage wurde ausgeführt.

```
-- Teil D Prüfung 1 Warehouse Tabellen auflisten
-- Erwartet Genau sechs Tabellen
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'brewflow_warehouse'
  AND table_type = 'BASE TABLE'
ORDER BY table_name;
```

Kontrolle Die Ergebnisliste enthält dim_channel, dim_customer, dim_date, dim_product, dim_store und fact_order_items.

Ergebnis Bestanden. Alle sechs erwarteten Tabellen sind vorhanden.

Abbildung 1 Ergebnisnachweis aus dem SQL-Client



Teil D Prüfung 2 Verkaufskanäle

Aufgabenstellung Die vorgegebenen Kanäle und ihre Gruppierung überprüfen.

Schritt Die Referenzdaten aus dim_channel nach Surrogatschlüssel sortiert anzeigen.

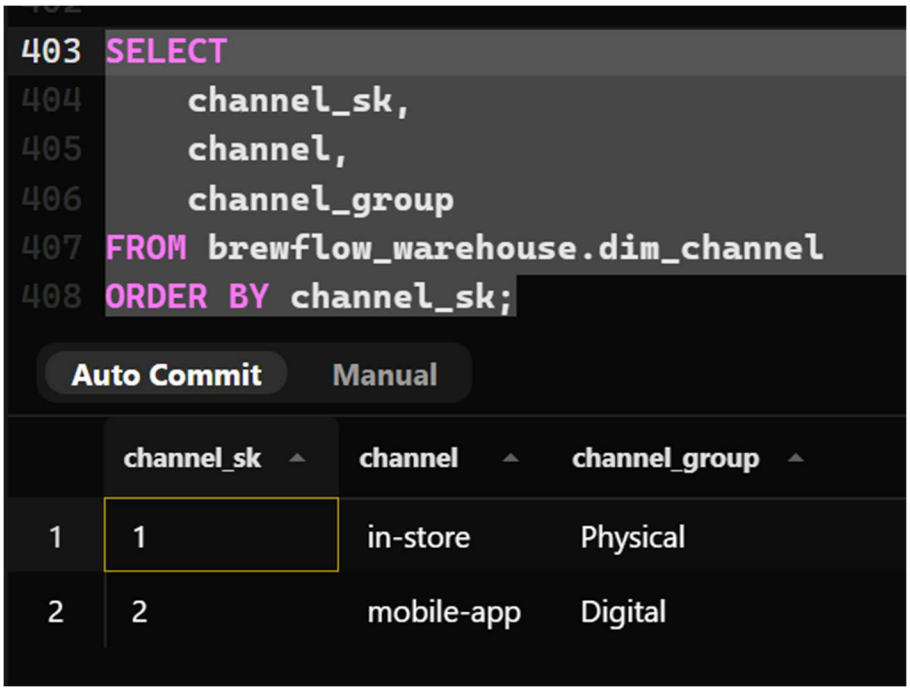
Aktion und SQL Die folgende Kontrollabfrage wurde ausgeführt.

```
-- Teil D Prüfung 2 Verkaufskanäle prüfen
SELECT channel_sk, channel, channel_group
FROM brewflow_warehouse.dim_channel
ORDER BY channel_sk;
```

Kontrolle Zwei Zeilen sind vorhanden: channel_sk 1 mit in-store und Physical sowie channel_sk 2 mit mobile-app und Digital.

Ergebnis Bestanden. Beide Verkaufskanäle sind korrekt befüllt.

Abbildung 2 Ergebnisnachweis aus dem SQL-Client



Teil D Prüfung 3 SCD Struktur

Aufgabenstellung Die vier vorgesehenen Strukturmerkmale der Kundendimension überprüfen.

Schritt Datentyp, NULL-Zulässigkeit, Identity-Eigenschaft und Standardwert der relevanten Spalten anzeigen.

Aktion und SQL Die folgende Kontrollabfrage wurde ausgeführt.

```
-- Teil D Prüfung 3 SCD Struktur prüfen
SELECT column_name, data_type, is_nullable,
       is_identity, column_default
FROM information_schema.columns
WHERE table_schema = 'brewflow_warehouse'
  AND table_name = 'dim_customer'
  AND column_name IN (
      'customer_sk', 'effective_from',
      'effective_to', 'is_current'
  )
ORDER BY ordinal_position;
```

Kontrolle customer_sk ist integer, NOT NULL und Identity. effective_from ist date und NOT NULL. effective_to ist date und erlaubt NULL. is_current ist boolean, erlaubt NULL und hat den Standardwert true.

Ergebnis Bestanden. Die SCD-Type-2-Struktur entspricht dem vorgesehenen DDL.

Abbildung 3 Ergebnisnachweis aus dem SQL-Client

	column_name	data_type	is_nullable	is_identity	column_default
1	customer_sk	integer	NO	YES	(NULL)
2	effective_from	date	NO	NO	(NULL)
3	effective_to	date	YES	NO	(NULL)
4	is_current	boolean	YES	NO	true

Teil D Prüfung 4 Dimensionsverweise

Aufgabenstellung Die fünf Fremdschlüssel der Faktentabelle bestätigen.

Schritt Die Fremdschlüssel mit ihren Zieltabellen und Zielspalten aus dem Datenbankkatalog auslesen.

Aktion und SQL Die folgende Kontrollabfrage wurde ausgeführt.

```
-- Teil D Prüfung 4 Fremdschlüssel prüfen
SELECT
    kcu.column_name AS fact_column,
    ccu.table_name AS dimension_table,
    ccu.column_name AS dimension_key
FROM information_schema.table_constraints tc
JOIN information_schema.key_column_usage kcu
    ON tc.constraint_catalog = kcu.constraint_catalog
   AND tc.constraint_schema = kcu.constraint_schema
   AND tc.constraint_name = kcu.constraint_name
JOIN information_schema.constraint_column_usage ccu
    ON tc.constraint_catalog = ccu.constraint_catalog
   AND tc.constraint_schema = ccu.constraint_schema
   AND tc.constraint_name = ccu.constraint_name
WHERE tc.table_schema = 'brewflow_warehouse'
  AND tc.table_name = 'fact_order_items'
  AND tc.constraint_type = 'FOREIGN KEY'
ORDER BY kcu.column_name;
```

Kontrolle channel_sk, customer_sk, date_sk, product_sk und store_sk verweisen jeweils auf den gleichnamigen Schlüssel der zugehörigen Dimension.

Ergebnis Bestanden. Die Faktentabelle besitzt alle fünf vorgesehenen Dimensionsverweise.

Abbildung 4 Ergebnisnachweis aus dem SQL-Client

	fact_column	dimension_table	dimension_key
1	channel_sk	dim_channel	channel_sk
2	customer_sk	dim_customer	customer_sk
3	date_sk	dim_date	date_sk
4	product_sk	dim_product	product_sk
5	store_sk	dim_store	store_sk

Teil D Prüfung 5 Generierte Umsatzspalte

Aufgabenstellung Die automatische Berechnung von revenue und den vorgesehenen Zahlentyp bestätigen.

Schritt Die Metadaten und den Generierungsausdruck der Spalte revenue auslesen.

Aktion und SQL Die folgende Kontrollabfrage wurde ausgeführt.

```
-- Teil D Prüfung 5 Generierte Umsatzspalte prüfen
SELECT column_name, data_type,
       numeric_precision, numeric_scale,
       is_generated, generation_expression
FROM information_schema.columns
WHERE table_schema = 'brewflow_warehouse'
  AND table_name = 'fact_order_items'
  AND column_name = 'revenue';
```

Kontrolle revenue ist numeric mit Präzision 8 und Skala 2. is_generated ist ALWAYS. Der Ausdruck ((quantity)::numeric * unit_price) entspricht Menge × Einzelpreis; die Typumwandlung ist korrekt.

Ergebnis Bestanden. revenue wird automatisch berechnet und beim Laden nicht manuell befüllt.

Abbildung 5 Ergebnisnachweis aus dem SQL-Client

	column_name	data_type	numeric_precision	numeric_scale	is_generated	generation_expression
1	revenue	numeric	8	2	ALWAYS	((quantity)::numeric * unit_price)

Abschluss von Teil 2

Alle geforderten Strukturprüfungen sind bestanden. Das Star-Schema ist bereit für die anschließenden Ladeprozesse. Die tatsächliche Historisierung der Kunden, die Kalenderbefüllung und das Laden der Fakten werden durch das DDL noch nicht ausgeführt.